

Sistema de Arquivos com Journaling

*Projeto desenvolvido em Java para a disciplina de Sistemas Operacionais.
Implementa um sistema de arquivos virtual com suporte a Journaling (WAL) para garantir integridade dos dados em caso de falhas.*

Integrantes do Projeto:

- Isadora Ferreira Neves Rios
- Cauan Gomes dos Santos Barbosa

<https://github.com/cauang/JournalingSO>

Sumário

1. O que é um Sistema de Arquivos?
 2. O que é Journaling?
 3. Tipos de Journaling
 4. Arquitetura do Projeto
 5. Estrutura de Dados
 6. Como o Journaling funciona neste projeto
 7. Descrição das Classes
 8. Como Instalar e Executar
 9. Comandos do Shell
 10. Testando o Journaling na prática
-

1. O que é um Sistema de Arquivos?

Um **sistema de arquivos** é a camada do sistema operacional responsável por organizar, armazenar e recuperar dados em um meio físico (HD, SSD, pendrive etc.). Sem ele, todos os dados em disco seriam apenas uma sequência contínua de bytes sem nenhuma estrutura lógica — seria impossível saber onde começa ou termina um arquivo.

O sistema de arquivos define:

- Como os arquivos são **nomeados** e organizados em **diretórios**
- Como o espaço em disco é **alocado e gerenciado**
- Quais **metadados** são armazenados (datas, permissões, tamanho)
- Como os dados são **lidos e escritos** com segurança

Exemplos reais de sistemas de arquivos: **ext4** (Linux), **NTFS** (Windows), **APFS** (macOS), **FAT32** (pendrives).

2. O que é Journaling?

Imagine que o sistema operacional precisa salvar um arquivo. Internamente, isso envolve várias etapas: atualizar o diretório, alocar blocos no disco, gravar os dados... Se a energia cair no meio desse processo, o sistema de arquivos fica em um **estado inconsistente** — o arquivo existe no diretório, mas os dados não foram gravados corretamente.

O **Journaling** resolve esse problema. Antes de fazer qualquer modificação real no sistema de arquivos, o SO registra a intenção da operação em uma área separada chamada **journal** (ou log). Assim:

- Se o sistema **cair antes** de concluir, o journal revela que a operação estava incompleta — ela é **ignorada** (Undo).
- Se o sistema **cair depois** de registrar o commit, o journal sabe que a operação foi concluída mas ainda não aplicada — ela é **reexecutada** (Redo).

Essa técnica é chamada de **Write-Ahead Logging (WAL)**: o log é escrito **antes** da operação ser aplicada.

3. Tipos de Journaling

Tipo	O que registra	Desempenho	Segurança
Write-Ahead Logging (WAL)	Operações antes de aplicar	Médio	Alta
Metadata Journaling	Só metadados (ext3/ext4/NTFS)	Alto	Média
Full Journaling	Metadados + dados reais	Baixo	Máxima
Log-Structured FS	Tudo como um log circular	Muito alto	Alta

Este projeto implementa o **Write-Ahead Logging**: cada operação é registrada no `journal.log` com um `START` e um `COMMIT`, e ao iniciar o programa, o log é verificado para recuperar operações que não chegaram a ser salvas em disco.

4. Arquitetura do Projeto

```
src/  
  Main.java          # Ponto de entrada  
model/  
  FSNode.java        # Classe base abstrata (arquivo ou diretório)  
  File.java          # Representa um arquivo  
  Directory.java     # Representa um diretório  
service/  
  FileSystemSimulator.java # Lógica do sistema de arquivos  
  Journal.java       # Gerenciamento do log (WAL)  
shell/  
  Shell.java         # Interface de linha de comando (CLI)  
  
data/                # Gerado automaticamente em tempo de execução  
  filesystem.db       # Estado do sistema (checkpoint serializado)  
  journal.log         # Log de transações (WAL)
```

Fluxo geral:

Usuário digita um comando

Shell interpreta e chama o `FileSystemSimulator`

`FileSystemSimulator` registra no Journal (START)

Operação é aplicada em memória (árvore de diretórios)

Journal registra o COMMIT

Na saída limpa (exit), o estado é serializado em `filesystem.db`

5. Estrutura de Dados

O sistema de arquivos é representado em memória como uma **árvore**:

```
/ (raiz - Directory)
  documentos/ (Directory)
    relatorio.txt (File)
    notas.txt (File)
  trabalhos/ (Directory)
    so.txt (File)
```

Cada nó da árvore é um `FSNode`, que pode ser:

- `Directory` : contém outros nós (filhos) em um `HashMap<String, FSNode>`
- `File` : contém o conteúdo textual do arquivo

Todos os nós guardam metadados: nome, referência ao pai e data de modificação.

O atributo `pai` é marcado como `transient` na serialização para evitar referências circulares. Ao carregar o checkpoint do disco, os pais são reconstruídos com o método `resolverPais()`.

6. Como o Journaling funciona neste projeto

Formato do log (`data/journal.log`)

Cada transação ocupa duas linhas:

```
1 | START | mkdir | L2RvY3VtZW50b3M=
1 | COMMIT | mkdir
```

- O primeiro campo é o **ID da transação**
- O segundo é o **tipo**: `START` ou `COMMIT`
- O terceiro é a **operação**: `mkdir`, `rm`, `mv`, `cp`, `write`
- Os campos seguintes são os **argumentos codificados em Base64** (para suportar espaços e caracteres especiais)

Processo de recuperação ao iniciar

1. Lê filesystem.db carrega o último estado salvo
2. Lê journal.log busca transações com START + COMMIT
3. Reexecuta cada transação encontrada (as operações perdidas)
4. Salva novo checkpoint e limpa o log

Se uma transação tiver `START` mas **não tiver** `COMMIT`, ela é ignorada — isso indica que o sistema caiu no meio da operação, então não há o que recuperar (o dado nunca foi confirmado).

7. Descrição das Classes

FSNode (model)

Classe abstrata base. Define os atributos comuns a arquivos e diretórios: `nome`, `pai`, `dataModificacao`. Contém o método `getCaminho()` que reconstrói o caminho absoluto recursivamente até a raiz.

File (model)

Estende `FSNode`. Representa um arquivo com conteúdo textual (`conteudo`). O tamanho é calculado em bytes com base no conteúdo.

Directory (model)

Estende `FSNode`. Representa um diretório que armazena outros nós em um `HashMap`. Possui métodos para adicionar, remover e buscar filhos.

Journal (service)

Gerencia o arquivo `data/journal.log`. Responsável por:

- Gerar IDs de transação sequenciais
- Escrever linhas `START` e `COMMIT` no log
- Ler e interpretar o log para encontrar transações pendentes
- Codificar argumentos em Base64 e decodificá-los na leitura

FileSystemSimulator (service)

Núcleo do sistema. Mantém a árvore de diretórios em memória e implementa todas as operações (`mkdir` , `rm` , `mv` , `cp` , `write` , `cd` , `ls` , `cat`). Cada operação de escrita:

1. Converte o caminho para absoluto
2. Registra no Journal (START)
3. Aplica a operação em memória
4. Registra o COMMIT no Journal

Também gerencia o checkpoint: serializa a árvore em `filesystem.db` e limpa o log ao salvar.

Shell (shell)

Interface interativa de linha de comando. Lê comandos do usuário, valida os argumentos e chama os métodos do `FileSystemSimulator` . O comando `crash` encerra a JVM abruptamente (`System.exit(99)`) sem salvar, permitindo testar a recuperação pelo Journal.

Main

Ponto de entrada. Inicializa o `Journal` , o `FileSystemSimulator` , executa o `replayJournal()` para recuperação e inicia o `Shell` .

8. Como Instalar e Executar

Requisitos

- **Java JDK 8 ou superior** instalado
- Terminal (Prompt de Comando, PowerShell ou Bash)

Compilar

```
javac -encoding UTF-8 -d out src/model/*.java src/service/*.java src/shell/*.java src/Main.java
```

Executar

```
java -cp out Main
```

As pastas `out/` e `data/` são criadas automaticamente. Não é necessário criá-las manualmente.

9. Comandos do Shell

Comando	Descrição
<code>help</code>	Mostra os comandos disponíveis
<code>ls [caminho]</code>	Lista arquivos e pastas
<code>cd <caminho></code>	Entra em um diretório
<code>mkdir <nome></code>	Cria um diretório
<code>touch <nome></code>	Cria um arquivo vazio
<code>write <arquivo> <texto></code>	Escreve texto em um arquivo
<code>cat <arquivo></code>	Exibe o conteúdo de um arquivo
<code>rm <caminho></code>	Remove um arquivo ou diretório
<code>mv <origem> <destino></code>	Move ou renomeia
<code>cp <origem> <destino></code>	Copia arquivo ou diretório
<code>checkpoint</code>	Força salvamento do estado em disco
<code>crash</code>	Simula falha abrupta (para testar journaling)
<code>exit</code>	Salva o estado e encerra o programa

10. Testando o Journaling na prática

Este é o teste mais importante para entender o journaling. Siga os passos:

Passo 1 — Criar arquivos e causar um crash

```
java -cp out Main
```

```
so-fs:/$ mkdir projetos
```

```
so-fs:/$ cd projetos
so-fs:/projetos$ write resumo.txt Conteudo importante que nao pode ser perdido
so-fs:/projetos$ ls
so-fs:/projetos$ crash
```

O programa encerra imediatamente **sem** salvar o estado em `filesystem.db` .

Passo 2 — Verificar o journal

Abra o arquivo `data/journal.log` e veja que as operações foram registradas:

```
1 | START | mkdir | L3Byb2pldG9z
1 | COMMIT | mkdir
2 | START | write | L3Byb2pldG9zL3Jlc3Vtby50eHQ= | Q29udGV1ZG8gaW1wb3J0YW50ZS4uI
2 | COMMIT | write
```

Passo 3 — Reiniciar e observar a recuperação

```
java -cp out Main
```

O simulador vai exibir automaticamente:

```
[JOURNAL] 2 transacoes pendentes. Recuperando...
Reexecutando: Tx#1 mkdir [/projetos]
Reexecutando: Tx#2 write [/projetos/resumo.txt, Conteudo importante...]
[JOURNAL] Recuperacao concluida. Salvando checkpoint...
```

Passo 4 — Confirmar que os dados estão intactos

```
so-fs:/$ ls
so-fs:/$ cat projetos/resumo.txt
```

Os dados foram recuperados completamente pelo journaling, mesmo sem um `exit` limpo.
